



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Intelligent Airport Runway Safety and Aircraft Ground Operations Platform

A CAPSTONE PROJECT REPORT

Submitted partial fulfilment for the Course of

CSA1016 – Software Engineering

to receive the award of the degree

of **BACHELOR OF**

ENGINEERING IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

A. GOVARDHAN REDDY (192421359)

Under the Supervision of

Dr. Anita Davamani k

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

JULY-2026

Table of Contents

S.No.	Contents	Page No.
1	Abstract	3
2	Industry Context	4
3	Problem Statement	4
4	Objectives	6
5	Review of Existing Systems and Related Work	6
6	Proposed System Architecture	8
7	Core System Modules	9
8	Technology Stack	11
9	Data Model and Database Design	12
10	Core Algorithms and Techniques	13
11	Implementation Strategy	14
12	Application of Advanced Methodologies and Agile Project Management	15
13	Testing Strategy	19
14	Expected Outcomes	20
15	Challenges and Risk Considerations	21
16	Future Scope	22
17	Conclusion	22
18	References	23

Abstract

Runway incursions and uncoordinated ground operations remain among the most persistent safety and efficiency risks in commercial aviation. As global air traffic volumes continue to rise, airports are under mounting pressure to strengthen airside situational awareness without proportionally scaling staffing or infrastructure costs. This case study presents the design and engineering approach for an Intelligent Airport Runway Safety and Aircraft Ground Operations Platform (IARSAGOP) — a unified, real-time software system that fuses aircraft surveillance data, ground vehicle telemetry, weather feeds, and historical incident records to deliver predictive hazard alerts, automated incursion detection, and coordinated ground-operations visibility.

The report is structured as a formal software engineering case study. It begins by examining the industry context and the operational shortcomings of legacy, fragmented runway-safety systems, then translates the business problem into concrete engineering objectives. It proceeds through requirements analysis, system architecture, module-level design, data modelling, core algorithms, technology stack selection, implementation strategy, testing methodology, and a discussion of expected outcomes, risks, and future scope.

The proposed platform is architected as a distributed, event-driven, microservices-based system that ingests data from Automatic Dependent Surveillance–Broadcast (ADS-B) receivers, Airport Surface Detection Equipment (ASDE-X/ASSC) radar, ground vehicle GPS transponders, and meteorological sensors. A real-time geofencing and trajectory-conflict engine continuously evaluates the airside environment, while a machine-learning-based predictive risk model scores the probability of a runway conflict before it materializes. Role-based dashboards deliver tailored situational views to air traffic controllers, ground crew supervisors, and airport safety officers, and a centralized incident-logging subsystem digitizes near-miss reporting to support long-term safety analytics.

The case study concludes that a well-engineered, modular, and horizontally scalable platform of this kind can materially reduce runway incursion risk, shorten aircraft turnaround times, and give airport safety teams a data-driven foundation for continuous improvement, while remaining deployable across airports of varying size and traffic complexity.

Industry Context

The aviation industry is governed by some of the most rigorous safety frameworks of any transportation sector, yet runway incursions — the unauthorized presence of an aircraft, vehicle, or person on a runway — continue to rank among the most serious threats to flight safety. Regulatory bodies such as the International Civil Aviation Organization (ICAO), the U.S. Federal Aviation Administration (FAA), and the European Union Aviation Safety Agency (EASA) classify runway safety as a top-tier risk category precisely because a single incursion event has the potential to result in catastrophic loss of life.

Modern airports are complex, multi-actor environments. At any given moment, a busy hub may be simultaneously coordinating incoming and outgoing flights, taxiing aircraft, fuel trucks, baggage tugs, catering vehicles, de-icing crews, and maintenance personnel — all sharing a constrained physical footprint. Historically, this coordination has depended on a combination of air traffic controller vigilance, VHF radio communication, and visual observation from control towers. While effective for decades, this model is inherently vulnerable to human factors: fatigue, radio congestion, non-standard phraseology, poor visibility, and the sheer cognitive load of tracking dozens of moving assets simultaneously.

Global air traffic has grown steadily, and forecasts from IATA and ACI World project continued passenger and cargo growth over the coming decade. Airports cannot scale controller headcount and apron staff at the same rate as traffic growth without significant cost pressure, which makes automation, sensor fusion, and predictive analytics an operational necessity rather than a convenience. At the same time, ground operations — pushback, refuelling, baggage handling, and turnaround servicing — are frequently managed by separate teams and separate systems from those used for airside surveillance, creating information silos that slow decision-making and obscure the root causes of delay.

The convergence of several technology trends now makes an integrated safety and operations platform feasible: low-cost ADS-B and multilateration surveillance, ubiquitous GPS-enabled ground vehicle telemetry, cloud-native event-streaming architectures, and mature machine-learning techniques for anomaly and risk prediction. This case study explores how these technologies can be combined into a single, coherent software platform purpose-built for airside safety and ground-operations coordination.

1. Problem Statement

Airport runway and ground operations currently rely on fragmented systems and manual coordination between air traffic control, ground crews, and vehicle operators, which elevates the risk of runway incursions and contributes to avoidable operational delays. Several structural gaps define the problem space that this platform is designed to close:

- Lack of real-time data integration — aircraft position data, ground vehicle movement, and weather conditions are typically monitored through separate, unlinked systems, which limits proactive hazard detection.
- High cognitive load on controllers — human operators must simultaneously track multiple aircraft and ground assets, increasing the probability of oversight during high-traffic or low-visibility periods.
- Reactive rather than predictive safety posture — most existing surveillance tools alert only after a conflict condition already exists, leaving little to no lead time for preventive intervention.
- Untracked ground-operations correlation — pushback delays, late fueling, and baggage-handling bottlenecks are not systematically linked to runway scheduling, producing cascading downstream flight delays.
- Manual incident and near-miss reporting — safety events are frequently logged on paper or in disconnected spreadsheets, which slows trend analysis and undermines data-driven safety improvement programs.

Taken together, these gaps point to the need for a single, unified platform that consolidates runway safety monitoring, aircraft and ground vehicle tracking, predictive risk alerting, and ground-operations coordination into one real-time intelligent system — the objective this case study addresses.

2. Objectives

Based on the problem statement, the platform is designed around the following engineering objectives:

- Develop a real-time aircraft and ground vehicle tracking module that consumes position and movement data across the entire airside environment.
- Design a runway incursion detection subsystem capable of flagging potential conflicts between aircraft, vehicles, and personnel in real time.
- Build a predictive risk alert engine that identifies high-probability hazard scenarios using traffic density, weather conditions, and historical incident data.
- Implement a ground operations coordination dashboard that tracks pushback, fueling, baggage handling, and turnaround timelines end-to-end.
- Create a centralized incident and near-miss logging system to replace manual reporting and support structured safety-trend analysis.
- Integrate live weather and visibility data to dynamically adjust safety thresholds and alert sensitivity.
- Provide role-based visualization dashboards tailored to air traffic controllers, ground crew supervisors, and airport safety officers.
- Ensure the platform architecture scales across airports of varying size, traffic density, and operational complexity.

3. Review of Existing Systems and Related Work

Before defining the proposed architecture, it is useful to survey the categories of systems already deployed in the industry and identify their limitations, since the proposed platform is designed to consolidate — rather than replace — much of this existing sensor infrastructure.

3.1 Airport Surface Detection Equipment (ASDE-X / ASSC)

Radar- and multilateration-based surface surveillance systems provide controllers with a radar-like display of aircraft and transponder-equipped vehicles on the airport surface. These systems are effective at raw position detection but are generally standalone: they do not natively incorporate

weather context, ground-operations timelines, or predictive risk scoring, and their alerts are largely threshold-based rather than probabilistic.

3.2 Automatic Dependent Surveillance–Broadcast (ADS-B)

ADS-B allows aircraft to broadcast GPS-derived position, velocity, and identity data, which has become a cornerstone of modern surveillance. However, ADS-B alone does not cover non-equipped ground vehicles or personnel, and its data is rarely fused programmatically with ground-operations or weather systems in legacy deployments.

3.3 Runway Status Lights (RWSL) and Similar Automated Warning Systems

Automated in-pavement lighting systems illuminate to warn pilots and vehicle operators of an occupied or unsafe runway. These systems are valuable last-line-of-defence tools but are purely reactive — they respond to a detected conflict rather than forecasting one — and they provide no analytics layer for airport management.

3.4 Airport Operations Databases and Turnaround Management Tools

Many airports use separate resource- and stand-management software for ground handling, often maintained by different vendors than the airside surveillance systems. This separation is precisely the information silo identified in the problem statement, and it is a primary justification for a unifying platform.

3.5 Gap Analysis

Across all of the above categories, three consistent gaps emerge: (1) surveillance, weather, and ground-operations data are rarely fused into a single real-time model; (2) predictive, ML-driven risk scoring is largely absent from deployed systems, which remain rule-based and reactive; and (3) incident and near-miss data capture remains manual in a large share of airports, limiting the feedback loop needed for continuous safety improvement. The platform proposed in this case study is designed specifically to close these three gaps through a consolidated, cloud-native software architecture.

4. Proposed System Architecture

The platform is designed as a distributed, event-driven, microservices-based system so that individual capabilities — tracking, conflict detection, prediction, dashboards, and incident logging — can be developed, deployed, and scaled independently. The architecture is organized into five logical layers.

4.1 Layered Architecture Overview

- Data Ingestion Layer — collects raw feeds from ADS-B/ASDE-X receivers, ground vehicle GPS transponders, personnel tracking beacons, and meteorological sensors.
- Stream Processing Layer — normalizes, timestamps, and publishes incoming data onto a distributed event bus for low-latency consumption by downstream services.
- Core Intelligence Layer — hosts the incursion-detection engine, the predictive risk-scoring model, and the ground-operations correlation engine.
- Application / Service Layer — exposes REST and WebSocket APIs for dashboards, alerting, and the incident-logging subsystem, and coordinates role-based access control.
- Presentation Layer — role-based web dashboards for controllers, ground crew supervisors, and safety officers, plus mobile-friendly views for apron personnel.

4.2 Architectural Style

A microservices approach was selected over a monolithic design for three reasons. First, the platform's core functions — real-time geofencing, ML-based prediction, ground-operations tracking, and incident management — have materially different scaling and latency profiles, and independent scaling reduces infrastructure cost. Second, microservices allow safety-critical detection logic to be isolated, tested, and certified separately from lower-criticality reporting and analytics features. Third, an event-driven backbone (a publish-subscribe message broker) naturally matches the platform's real-time, sensor-fusion nature, since every new position report, weather update, or ground-status change is simply another event on the bus.

4.3 High-Level Data Flow

Position reports from aircraft and ground vehicles, together with weather updates, are published as discrete events onto the message bus. The Incursion Detection Service and the Predictive

Risk Engine both subscribe to this stream, each maintaining an in-memory spatial model of the airside environment. Detected conflicts and elevated risk scores are pushed to the Alerting Service, which fans out notifications to the relevant role-based dashboards over WebSocket connections and, where configured, to physical alerting systems such as runway status lights. In parallel, the Ground Operations Service consumes turnaround-milestone events (pushback request, fueling start/end, baggage completion) and correlates them against the runway schedule to flag potential delay cascades. All safety-relevant events are additionally persisted to the Incident and Analytics Data Store for trend analysis.

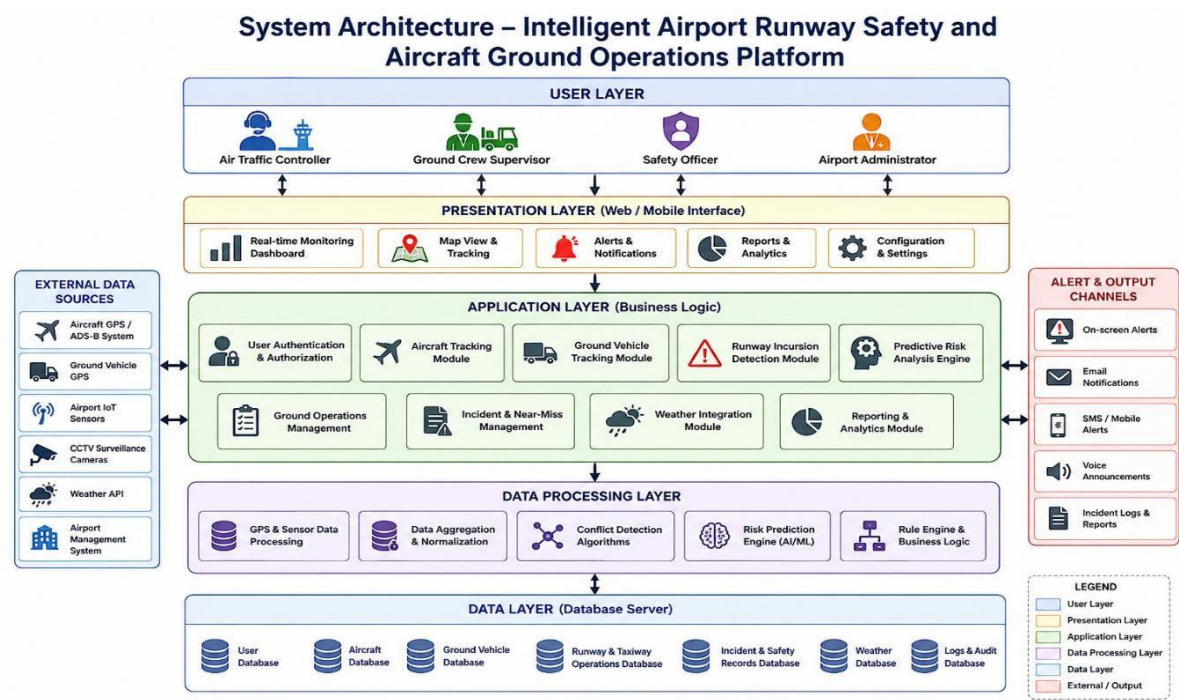


Fig 1: System Architecture

5. Core System Modules

5.1 Real-Time Aircraft and Ground Vehicle Tracking Module

This module ingests position, velocity, and heading data from ADS-B receivers, surface-movement radar, and GPS-equipped ground vehicle transponders. Each tracked object is represented internally as a lightweight state record containing a unique identifier, object class (aircraft, tug, fuel truck, personnel beacon, etc.), latitude/longitude, altitude where relevant, heading, speed, and a last-updated timestamp. The module normalizes disparate feed formats into a single internal schema and republishes a unified track stream that all downstream services consume, which is what allows the platform to reason about aircraft and ground vehicles within one coherent spatial model rather than as separate silos.

5.2 Runway Incursion Detection Engine

The detection engine maintains a digital twin of the airport's runways, taxiways, and safety-critical hold-short zones as geofenced polygons. On every incoming track update, the engine performs point-in-polygon and trajectory-intersection checks to determine whether an aircraft, vehicle, or person is present in, or projected to enter, a protected zone without clearance. Conflicts are classified by severity — for example, a stationary vehicle near a hold line versus an aircraft and a vehicle on converging trajectories toward the same runway segment — and are escalated to the Alerting Service with the appropriate priority.

5.3 Predictive Risk Alert Engine

Where the detection engine identifies conditions that already constitute a conflict, the predictive engine estimates the probability of a future conflict before it occurs. It combines traffic density (number of active movements per runway per unit time), current and forecast weather (visibility, crosswind, precipitation), and historical incident patterns for the specific airport layout to produce a continuously updated risk score per runway and per active movement. When the score crosses a configurable threshold, the engine raises an early advisory, giving controllers additional lead time that reactive, threshold-only systems cannot provide.

5.4 Ground Operations Coordination Dashboard

This module tracks the full turnaround lifecycle of each aircraft — pushback request and execution, fueling start and completion, baggage and cargo handling, catering, and de-icing where applicable — against a target turnaround timeline. Delays at any stage are automatically flagged and cross-referenced against the runway departure schedule, allowing ground supervisors and controllers to see, in one place, when a ground-side delay is likely to cascade into a runway-scheduling conflict.

5.5 Centralized Incident and Near-Miss Logging System

Replacing manual, paper-based reporting, this module provides a structured digital form for logging incidents and near-misses, automatically pre-populated with relevant track data, timestamps, and environmental conditions captured at the time of the event. Logged incidents feed directly into the analytics data store, enabling safety officers to run trend analysis without a manual data-entry step.

5.6 Weather and Visibility Integration

The platform ingests live meteorological data — wind speed and direction, visibility, precipitation, and runway surface condition reports — and uses it to dynamically adjust the sensitivity of both the incursion-detection thresholds and the predictive risk model. For example, alert thresholds are automatically tightened during low-visibility operations, reflecting the elevated baseline risk of those conditions.

5.7 Role-Based Visualization Dashboards

Three primary dashboard views are provided. The Air Traffic Controller view presents a consolidated radar-style map of aircraft and ground assets with active alerts overlaid. The Ground Crew Supervisor view emphasizes turnaround timelines, task status, and delay flags across the apron. The Airport Safety Officer view surfaces incident trends, near-miss statistics, and risk-engine performance metrics for audit and continuous-improvement purposes.

5.8 Scalability and Multi-Airport Support

The platform is designed to be configuration-driven rather than hard-coded to a single airport layout, so that runway and taxiway geofences, sensor integrations, and operational thresholds can be defined per site. This allows the same core services to be deployed at a small regional airfield or a

major international hub, with resource allocation (compute, message-bus partitions, database sharding) scaled to the traffic volume of each deployment.

6. Technology Stack

The technology choices below reflect the platform's dual requirement for hard real-time performance in the detection path and rich analytical capability in the reporting path.

Layer / Concern	Recommended Technologies
Data Ingestion	MQTT / AMQP listeners, ADS-B decoders (dump1090-class libraries), custom GPS telemetry adapters
Event Streaming	Apache Kafka (or equivalent managed event bus) for high-throughput, low-latency track and sensor events
Stream Processing	Apache Flink / Kafka Streams for windowed spatial computations and real-time aggregation
Backend Services	Java (Spring Boot) or Go for latency-sensitive detection services; Python (FastAPI) for the ML risk-scoring service
Machine Learning	Python, scikit-learn / XGBoost for risk scoring, with TensorFlow or PyTorch reserved for future trajectory-prediction models
Geospatial Engine	PostGIS extensions and a spatial-indexing library (e.g. R-tree/S2) for geofence and trajectory-intersection queries
Primary Database	PostgreSQL with PostGIS for relational and spatial data (airport layout, incidents, turnaround records)
Time-Series Store	TimescaleDB or InfluxDB for high-frequency position and sensor telemetry
Caching Layer	Redis for in-memory active-track state and low-latency lookups
API Layer	REST for configuration and reporting endpoints; WebSocket/gRPC-streaming for real-time dashboard updates
Frontend	React with a mapping library (e.g. Mapbox GL / deck.gl) for real-time geospatial visualization
Infrastructure	Docker containers orchestrated with Kubernetes; cloud deployment (AWS/Azure/GCP) with autoscaling
Monitoring & Observability	Prometheus and Grafana for system health; the ELK/OpenSearch stack for centralized logging
Security	OAuth2/OIDC-based role-based access control, TLS in transit, encryption at rest for incident and personal data

Table 1: Recommended technology stack by architectural layer

7. Data Model and Database Design

The data model is split between a relational/spatial store for structured entities (runway geometry, incidents, turnaround records, user accounts) and a time-series store for high-frequency telemetry (raw position reports, weather snapshots). This separation keeps write-heavy telemetry ingestion from degrading the performance of analytical and reporting queries. The core relational entities are summarized below.

Entity	Key Attributes
Aircraft_Track	track_id, callsign, lat, lon, altitude, heading, speed_kts, source, timestamp
Ground_Vehicle_Track	vehicle_id, vehicle_type, lat, lon, heading, speed_kts, assigned_crew_id, timestamp
Runway_Zone	zone_id, airport_id, zone_type (runway/taxiway/hold-short), polygon_geometry, active_status
Incursion_Event	event_id, zone_id, involved_track_ids, severity, detected_at, resolved_at, alert_status
Risk_Score	score_id, runway_id, timestamp, traffic_density, weather_factor, historical_factor, composite_score
Turnaround_Record	flight_id, aircraft_id, pushback_time, fuel_start, fuel_end, baggage_complete, target_departure
Weather_Snapshot	snapshot_id, airport_id, wind_speed, wind_dir, visibility, precipitation, recorded_at
Incident_Report	report_id, reported_by, event_type, description, related_event_id, created_at, status
User_Account	user_id, role (controller/supervisor/safety_officer), airport_id, credentials_hash, permissions

Table 2: Core data entities and representative attributes

Spatial relationships — such as whether a track point falls within a runway or hold-short polygon — are resolved using PostGIS spatial indexing, while high-frequency raw telemetry is retained in a time-series store and periodically down-sampled into the relational layer for long-term trend analysis, keeping storage costs proportionate to the operational value of the data over time.

8. Core Algorithms and Techniques

8.1 Geofencing and Point-in-Polygon Conflict Checks

Each runway, taxiway, and hold-short line is modelled as a polygon. Incoming track points are tested against these polygons using a spatial-indexing structure (such as an R-tree) to keep lookups efficient even as the number of tracked objects grows, avoiding a brute-force check against every zone for every update.

8.2 Trajectory Conflict Prediction

Beyond current position, the engine linearly extrapolates each track's short-term trajectory using its current heading and speed, then checks whether two extrapolated paths converge within a protected zone inside a defined time horizon (for example, the next 30–60 seconds). This allows the system to flag a developing conflict before either object physically enters the zone.

8.3 Predictive Risk Scoring Model

The predictive risk engine combines multiple weighted factors — current traffic density on the runway, live weather severity, and a historical incident-frequency factor for similar conditions at that airport — into a composite risk score. A gradient-boosted tree model (or a simpler logistic-regression baseline for smaller deployments) is trained on historical incident and near-miss data to learn the relative importance of these factors, and is periodically retrained as new incident data accumulates.

8.4 Dynamic Threshold Adjustment

Alert sensitivity is not static: visibility and precipitation values from the Weather Integration module are used to scale detection and risk-score thresholds, so that, for instance, a lower composite risk score triggers an advisory during low-visibility operations than would be required in clear conditions.

8.5 Delay-Cascade Correlation

The Ground Operations module compares actual turnaround milestone timestamps against target timelines and the published runway schedule, using a simple rule-based correlation (with room for a future ML-based estimator) to flag when an accumulating ground delay is likely to push a departure into a conflicting runway slot.

9. Implementation Strategy

Given the platform's safety-critical nature, an incremental, phased implementation approach is recommended over a single large release.

Phase 1 — Foundation and Data Ingestion

Establish the event-streaming backbone, integrate ADS-B and ground-vehicle telemetry feeds, and build the unified tracking data model. This phase focuses purely on reliable, low-latency data ingestion with no detection logic yet active.

Phase 2 — Core Detection Engine

Implement runway and taxiway geofencing, point-in-polygon conflict checks, and basic threshold-based alerting. This phase delivers the platform's minimum viable safety functionality and can be validated against recorded historical track data before any live deployment.

Phase 3 — Predictive Analytics and Weather Integration

Introduce the machine-learning risk-scoring model, historical incident data pipelines, and dynamic threshold adjustment based on live weather. This phase moves the platform from reactive to preventive.

Phase 4 — Ground Operations and Incident Logging

Build the ground-operations coordination dashboard and the centralized incident/near-miss logging subsystem, and connect delay-cascade correlation to the runway schedule.

Phase 5 — Role-Based Dashboards and Scale-Out

Finalize the controller, supervisor, and safety-officer dashboard views, and generalize configuration so the platform can be deployed at additional airports of varying size.

Throughout all phases, safety-critical detection code paths are developed under stricter code-review, testing, and change-control discipline than lower-criticality reporting features, in line with standard aviation software assurance practice.

10. Application of Advanced Methodologies and Agile Project Management

Delivering a safety-critical, multi-module platform such as IARSAGOP requires more than good architecture — it requires a project management approach that can absorb changing airport requirements, coordinate multiple specialist teams (tracking, ML, ground-operations, dashboards, DevOps), and still satisfy the documentation and verification discipline expected of aviation software. This section describes how advanced software development methodologies and agile project management techniques are applied to plan, execute, and govern the delivery of this project.

10.1 Methodology Selection: A Hybrid Agile–V-Model Approach

A pure Waterfall approach is poorly suited to this project because airport requirements (sensor availability, airport layout, integration constraints) are only fully understood once early increments are demonstrated to stakeholders, and a purely Agile approach with no formal verification stage would be inadequate for the safety-critical detection and prediction components. The project therefore adopts a hybrid model: Agile Scrum is used to iteratively design, build, and demonstrate the platform's features in short cycles, while safety-critical modules (incursion detection, predictive risk engine) additionally pass through V-Model-style formal verification and validation gates before being promoted to production. This hybrid pattern — sometimes called 'Agile with a safety overlay' — preserves the responsiveness of Agile for the majority of the system while meeting the rigor aviation stakeholders expect for the modules that directly influence runway-safety decisions.

10.2 Scrum Framework Applied to the Project

The core development effort is organized as Scrum, run in two-week sprints, with the following roles and responsibilities mapped to the project context:

Scrum Role	Project Mapping	Responsibility
Product Owner	Airport safety program lead / domain SME	Owns and prioritizes the product backlog based on safety impact, regulatory deadlines, and airport stakeholder needs
Scrum Master	Delivery / Agile coach	Facilitates ceremonies, removes impediments, and safeguards the team's ability to meet sprint commitments

Scrum Role	Project Mapping	Responsibility
Development Team	Backend, frontend, ML, and DevOps engineers	Cross-functional group that designs, builds, tests, and delivers a potentially shippable increment each sprint
Safety/QA Lead	Aviation software assurance specialist	Embedded in the team to review safety-critical increments against verification and traceability requirements
Stakeholder Group	ATC representatives, ground-ops managers, regulators	Participates in sprint reviews to validate increments against real operational needs

Table 3: Scrum roles mapped to the IARSAGOP project

Each sprint follows the standard Scrum ceremony cadence: Sprint Planning to select and commit to backlog items; a daily 15-minute Stand-up to surface blockers across the tracking, detection, ML, and dashboard sub-teams; a Sprint Review with airport stakeholders to demonstrate the working increment and gather feedback; and a Sprint Retrospective to continuously improve team process, tooling, and collaboration.

10.3 Product Backlog and Epic Structure

The product backlog is organized into epics that map directly to the platform's core modules, allowing the Product Owner to prioritize by safety impact and stakeholder value rather than by technical convenience:

- Epic 1 — Real-Time Tracking Ingestion (ADS-B, ground-vehicle GPS, personnel beacons)
- Epic 2 — Runway Incursion Detection Engine (geofencing, trajectory-conflict checks)
- Epic 3 — Predictive Risk Alert Engine (ML risk scoring, dynamic thresholds)
- Epic 4 — Ground Operations Coordination Dashboard (turnaround tracking, delay correlation)
- Epic 5 — Incident and Near-Miss Logging System
- Epic 6 — Role-Based Dashboards and Visualization
- Epic 7 — Platform Scalability, Multi-Airport Configuration, and DevOps Infrastructure

Each epic is decomposed into user stories with clear acceptance criteria (for example: 'As an air traffic controller, I want to see a real-time alert when a ground vehicle enters an active runway hold-

short zone, so that I can intervene before a conflict occurs'), which are estimated using Planning Poker and story points to build a predictable team velocity over successive sprints.

10.4 Release Roadmap Mapped to Agile Increments

The five implementation phases described earlier in this report are delivered as a sequence of Agile releases rather than as monolithic waterfall phases, so that stakeholders see working software early and often:

- Release 1 (Sprints 1–3): Foundation and Data Ingestion — deployable increment delivering live track visualization with no detection logic yet active.
- Release 2 (Sprints 4–6): Core Detection Engine — deployable increment adding geofencing and threshold-based incursion alerts, validated through safety replay-testing gates before release.
- Release 3 (Sprints 7–9): Predictive Analytics and Weather Integration — increment introducing the ML risk-scoring service and dynamic threshold adjustment.
- Release 4 (Sprints 10–12): Ground Operations and Incident Logging — increment delivering the coordination dashboard and digital incident-reporting workflow.
- Release 5 (Sprints 13–15): Role-Based Dashboards and Scale-Out — final increment generalizing configuration for multi-airport deployment.

Each release is preceded by a Release Planning session and followed by a stakeholder demo, ensuring that airport operations staff and safety officers can influence the backlog before the next release begins, which is a core advantage of Agile over a fixed, upfront specification.

10.5 Scaling Agile Across Multiple Teams (SAFe-Style Coordination)

Because the platform spans several specialist domains — real-time systems, machine learning, geospatial engineering, and enterprise dashboards — a Scaled Agile Framework (SAFe)-style structure is used to coordinate multiple Scrum teams. Each domain runs its own Scrum team and backlog, while a Program Increment (PI) Planning session held every 8–10 weeks aligns all teams around shared release objectives, cross-team dependencies (for example, the ML team's risk-score API contract needed by the dashboard team), and integration milestones. A lightweight Scrum-of-Scrums, held two to three times per week among team representatives, surfaces cross-team blockers well before they threaten a Program Increment deadline.

10.6 Kanban for Operations, Support, and Certification Workstreams

Not all project work fits a fixed two-week sprint cadence. Production support, regulatory documentation, and safety-certification evidence gathering are managed instead through a Kanban board with WIP (work-in-progress) limits, since this work arrives unpredictably and must be pulled through a continuous flow rather than batched into sprints. This blended Scrum-plus-Kanban approach — often called 'Scrumban' — lets the core feature teams keep sprint predictability while operational and compliance work is still tracked and prioritized transparently.

10.7 Agile Risk Management Techniques

Risk is managed continuously rather than only at project kickoff. A living risk register is reviewed at each Sprint Retrospective and Program Increment Planning session, with risks such as sensor-integration delays, ML model accuracy shortfalls, or third-party API instability scored by likelihood and impact and assigned an owner and mitigation action. High-risk or high-uncertainty backlog items (for example, the trajectory-conflict prediction algorithm) are addressed early through dedicated 'spike' stories — timeboxed research and prototyping efforts that reduce technical uncertainty before full implementation is committed to a sprint.

10.8 DevOps, CI/CD, and Toolchain

Agile delivery is supported by a DevOps toolchain that automates build, test, and deployment so that every sprint genuinely produces a releasable increment rather than an untested one:

Capability	Tooling	Purpose
Backlog & Sprint Management	Jira / Azure DevOps	Epics, user stories, sprint boards, and burndown tracking
Collaboration & Documentation	Confluence / SharePoint	Living documentation, architecture decision records, ceremony notes
Version Control	Git (GitHub/GitLab) with trunk-based or Gitflow branching	Source control with mandatory peer review on safety-critical modules
CI/CD Pipeline	Jenkins / GitHub Actions / GitLab CI	Automated build, unit test, and static-analysis gates on every commit
Continuous Deployment	Kubernetes with blue-green or canary rollout	Progressive, low-risk release of new increments to staging and production
Test Automation	JUnit/PyTest, Selenium/Cypress, custom replay-test harness	Automated regression and safety replay-testing integrated into the pipeline
Metrics & Dashboards	Jira/Azure DevOps burndown & velocity charts, Grafana	Sprint velocity, cycle time, defect leakage, and production health in one place

Table 4: Agile/DevOps toolchain supporting iterative delivery

10.9 Metrics, Transparency, and Continuous Improvement

Team and program health are tracked through sprint velocity, burndown/burnup charts, cycle time, and defect-leakage rate, all visible on shared dashboards so that the Product Owner and stakeholders have real-time visibility into progress without requiring status-report overhead. Retrospective action items are tracked to closure the same way backlog items are, ensuring that process improvements — not just product features — are treated as first-class, trackable work.

Taken together, this hybrid Agile–V-Model methodology, Scrum/SAFe/Kanban blend, and supporting DevOps toolchain allow the project to remain responsive to evolving airport requirements while still delivering the rigor, traceability, and verification discipline required of a system that directly supports runway-safety decision-making.

11. Testing Strategy

11.1 Unit and Component Testing

Each service — tracking ingestion, geofence evaluation, risk scoring, and delay correlation — is unit-tested in isolation, with particular emphasis on edge cases in the geospatial logic (objects on zone boundaries, rapid heading changes, sensor dropouts).

11.2 Integration Testing

Integration tests validate the full event pipeline end-to-end: a simulated position update should flow from ingestion, through the event bus, into the detection engine, and out to the alerting service within the platform's target latency budget.

11.3 Replay Testing Against Historical Data

Before any live deployment, the detection and predictive engines are validated by replaying anonymized historical track and incident data, comparing the platform's flagged events against the airport's actual recorded incidents and near-misses to measure detection accuracy and false-alarm rate.

11.4 Load and Latency Testing

Given the real-time nature of the platform, load tests simulate peak-traffic conditions — the maximum expected number of simultaneous aircraft and ground vehicle tracks — to confirm that end-to-end alert latency remains within the safety-relevant threshold under worst-case load.

11.5 User Acceptance Testing

Controllers, ground supervisors, and safety officers participate in structured UAT sessions using the role-based dashboards, with feedback used to refine alert presentation, dashboard layout, and alerting workflows before production rollout.

12. Expected Outcomes

- A significant reduction in runway incursion risk through real-time conflict detection and alerting.
- Improved situational awareness for air traffic controllers via a consolidated visual tracking view of aircraft and ground assets.
- Faster identification and resolution of ground-operation delays, reducing cascading flight-schedule disruptions.
- Enhanced predictive safety capability, enabling preventive action before incidents occur rather than purely reactive response.
- Streamlined incident reporting that enables data-driven safety audits and continuous-improvement programs.
- Reduced cognitive load on ground and air traffic personnel through automated monitoring and alerting.
- Better coordination across ground-crew teams, leading to improved aircraft turnaround efficiency.
- A scalable safety framework adaptable to airports of different sizes, supporting industry-wide adoption.

13. Challenges and Risk Considerations

13.1 Data Quality and Sensor Reliability

The platform's effectiveness is bounded by the quality and coverage of its input sensors. Gaps in ADS-B or vehicle-transponder coverage, or intermittent connectivity, can create blind spots that must be explicitly surfaced to controllers rather than silently degrading detection.

13.2 False Positives and Alert Fatigue

An overly sensitive detection or prediction engine risks generating excessive alerts, which can desensitize controllers over time. Threshold tuning, informed by replay testing against historical data, is essential to balance sensitivity against alert fatigue.

13.3 Latency and Availability Requirements

Because alerts are safety-relevant, the platform must meet strict end-to-end latency targets and high-availability guarantees; any architectural component on the detection critical path must be designed with redundancy and graceful degradation in mind.

13.4 Regulatory Certification and Change Control

Software that influences runway-safety decisions is likely to fall under aviation software assurance guidance (such as DO-178C-aligned practices), which imposes rigorous documentation, verification, and change-management requirements beyond typical commercial software development.

13.5 Integration with Legacy Airport Systems

Many airports operate legacy surveillance and ground-handling systems with limited or proprietary interfaces. Building resilient adapters for these systems, without requiring a full rip-and-replace of existing infrastructure, is a practical prerequisite for adoption.

13.6 Data Privacy and Security

Ground personnel tracking and incident reports may contain personal data, requiring the platform to implement role-based access control, encryption, and data-retention policies consistent with applicable privacy regulations.

14. Future Scope

- Extending the predictive risk model with deep-learning-based trajectory forecasting for more accurate multi-object conflict prediction.
- Incorporating computer-vision analysis from airfield cameras to detect uncooperative objects (wildlife, debris, unauthorized personnel) not visible to transponder-based tracking.
- Adding automated, closed-loop integration with runway status lighting and ground-vehicle alerting systems for faster response to detected conflicts.
- Building a cross-airport analytics layer that aggregates anonymized incident trends to support industry-wide benchmarking and safety research.
- Exploring digital-twin simulation capability so airports can model the safety impact of layout changes or new traffic patterns before implementation.

15. Conclusion

The Intelligent Airport Runway Safety and Aircraft Ground Operations Platform addresses a well-documented and persistent gap in airside safety management: the absence of a single, real-time, predictive system that unifies aircraft tracking, ground-vehicle movement, weather conditions, and ground-operations coordination. By combining an event-driven microservices architecture, geospatial conflict detection, machine-learning-based predictive risk scoring, and role-based dashboards, the platform is designed to shift airports from a largely reactive safety posture toward a proactive, data-driven one. This case study has outlined the industry context, the specific operational gaps that motivate the platform, a layered system architecture, detailed module designs, a supporting data model, core algorithms, a phased implementation strategy, and a testing approach appropriate to a safety-critical aviation system. While meaningful engineering and regulatory challenges remain — particularly around sensor reliability, alert-fatigue management, and certification — the overall approach is technically feasible with current sensor, streaming, and machine-learning technology, and offers a scalable path toward measurably safer and more efficient airport operations.

References

1. International Civil Aviation Organization (ICAO), Annex 14 — Aerodromes, and associated runway safety guidance material.
2. U.S. Federal Aviation Administration (FAA), Runway Safety Program documentation and Airport Surface Detection Equipment (ASDE-X) technical overviews.
3. EUROCONTROL, European Action Plan for the Prevention of Runway Incursions.
4. RTCA DO-260B, Minimum Operational Performance Standards for 1090 MHz Extended Squitter ADS-B.
5. RTCA DO-178C, Software Considerations in Airborne Systems and Equipment Certification (referenced for software assurance context).
6. IATA and ACI World, Annual Air Traffic and Airport Operations Statistical Reports.
7. Academic and industry literature on geofencing, spatial indexing, and event-driven microservice architectures for real-time tracking systems.
8. Scrum.org / Scrum Alliance, The Scrum Guide, for Scrum roles, artifacts, and ceremony definitions.
9. Scaled Agile, Inc., SAFe (Scaled Agile Framework) reference guidance for multi-team Program Increment planning.
10. Project Management Institute (PMI), Agile Practice Guide, for hybrid Agile-plan-driven project management practices.